

PAccel

A dependency-free quantized model container

Technical Specification & Delivery Proposal

Exact quantization algebra, container format, and a two-month plan to ship a bare-minimum port with no onnxruntime dependency.

$\approx 8\times$ weight-memory reduction at 4-bit	0 onnxruntime / protobuf deps	\$10k fixed delivery budget	8 wks to production handoff
--	--	---------------------------------------	---------------------------------------

Prepared by	Cybertwip — PowerEngine inference team
Prepared for	Microsoft — Windows AI / on-device inference
Document	PAccel format algebra, algorithms, and port delivery plan
Status	Proposal · Confidential

Contents

1. Executive summary
2. Strategic opportunity for Microsoft
3. What PAccel is
4. The quantization algebra (core)
5. Bit-packing and memory layout
6. Scale storage and fp16 numerics
7. Mixed-precision policy
8. Auxiliary codecs (int8 activations, RLE, v7 aliases)
9. The PACONNX1 container format
10. The compression pipeline and algorithms
11. The bare-minimum port (deliverable)
12. The V7 direct graph — the optimal inference route
13. NVIDIA DGX Spark — mixed CPU/GPU advantages
14. Reference architecture and Microsoft integration
15. Verification and quality
16. Delivery plan and timeline
17. Budget
18. Risks and mitigations
19. IP, licensing and security
20. The compute engine and native multimodal engines
- A. Appendix — symbols, worked example, file manifest

1 · Executive summary

PAccel is a compact, self-describing container that stores the weights of a neural network in block-wise quantized form. It was built inside the PowerEngine inference stack to let large language and diffusion models run on commodity hardware with a fraction of their original memory footprint. A 4-bit PAccel package holds roughly one eighth of the bytes of the original 32-bit checkpoint, and about one quarter of a 16-bit checkpoint, while preserving output quality through a smart mixed-precision policy that protects the handful of tensors that matter most.

This document does two things. First, it specifies the **exact algebra and algorithms** that produce the format: the block-wise quantizer, the bit-packing scheme, the half-precision scale encoding, the mixed-precision selection rule, the auxiliary codecs, and the byte-level container layout. Second, it proposes a **two-month, fixed-budget engagement** to deliver a bare-minimum, dependency-free port of the format to Microsoft: a Go library (`libpaccel`), a minimal extracted C++ accelerator, and a simple chatbot frontend with Hugging Face model-download capabilities — all of it building and running with **no onnxruntime dependency**.

The central engineering observation is that onnxruntime was never required by the format itself. In the existing engine it was reached only through a thin dynamic-load bridge for an optional accelerated matrix multiply. The quantization algebra, the container, and the decode path are pure arithmetic. Removing the runtime therefore costs nothing in capability and yields a tiny, portable, auditable codebase that Microsoft can embed directly into Windows AI experiences.

The ask

A fixed fee of **USD \$10,000** over **eight weeks** delivers the specification, the ported library and frontend, a cross-language parity test suite, and a packaged Windows demo. The budget is dominated by AI-augmented engineering labour, with a small allocation for AI tooling and cross-platform continuous-integration compute. Sections 16 and 17 give the milestone plan and the line-item breakdown.

2 · Strategic opportunity for Microsoft

Microsoft is moving inference to the edge. Copilot+ PCs, the Windows Copilot Runtime, Phi-class small language models, and DirectML all point toward a future where capable models run locally for latency, privacy, offline availability, and cost. The binding constraint on that future is memory: a model that does not fit in a laptop's RAM or NPU working set cannot run, no matter how fast the accelerator. Weight quantization is the lever that closes the gap, and a clean, portable container for quantized weights is the connective tissue between a downloaded checkpoint and an on-device runtime.

PAccel occupies a deliberately narrow, valuable niche. It is not a runtime and not a graph format; it is a **weight container plus a precise quantization contract**. That separation is its strength — it can feed DirectML, ONNX Runtime, a WebGPU path, or a pure-CPU kernel equally well, because it only promises to hand back correctly dequantized tensors.

Dimension	GGUF	ONNX (+ORT)	PAccel
Primary role	Weight container	Graph + weights	Weight container

Dimension	GGUF	ONNX (+ORT)	PAccel
Runtime dependency	llama.cpp family	onnxruntime	None (pure arithmetic)
Quantization contract	Many block types	QDQ / external	One block-wise rule, 2–8 bit
Footprint of the codec	Moderate	Large (100s of MB)	Tiny (single small library)
Mixed precision	Per-type	Manual	Automatic, name + shape aware
Cross-language parity	C/C++	C++/C#	Go and C++, bit-identical

Table 2.1 — Positioning. PAccel trades breadth for a tiny, auditable footprint and a single, well-specified quantization contract.

For Microsoft the attraction is supply-chain and footprint: a few thousand lines of dependency-free code with an exhaustively specified numeric contract is something a platform team can read, certify, and ship inside an operating system. The proposed port is engineered precisely to that bar.

3 · What PAccel is

A PAccel package is a single file beginning with the eight-byte signature PACONNX1, followed by a small header and a sequence of tensor records. Each record stores one weight tensor in one of four encodings: block-wise quantized (linear layout), block-wise quantized (AVX-friendly tiled int4 layout), raw, or run-length-encoded raw. The format ingests weights from three sources — ONNX initializers, the OnnxStream loose-weight manifest, and safetensors — and normalizes them into the same record structure.

The design rests on three ideas that the rest of this document makes precise. **First**, quantization is performed in small contiguous blocks, each with its own scale, so a single outlier weight only inflates the step size of its own block rather than the whole tensor. **Second**, the code map is symmetric with an implicit zero point, so dequantization is a single subtract-and-multiply and zero maps to zero exactly. **Third**, a small set of quality-critical tensors is automatically promoted to a higher bit width, because the cost of quantizing them aggressively is far larger than their share of the byte budget.

4 · The quantization algebra

This section gives the complete arithmetic that turns a tensor of floating-point weights into the packed codes and scales stored in a package. It is the heart of the format; everything else is bookkeeping around it.

4.1 Notation and the data model

Let a tensor be a flat sequence of real-valued weights $w_0, w_1, \dots, w_{N-1}$ taken in row-major order from its declared shape. Quantization is governed by two integer parameters chosen per package: the **bit width** b , an integer from 2 to 8, and the **block size** B , a positive integer that defaults to 32. From b we derive two derived constants used throughout:

```
maxCode = 2b - 1 (the largest representable code)
C = floor( maxCode / 2 ) (the center / implicit zero point)
```

For the default 4-bit profile, $\text{maxCode} = 15$ and $C = 7$. For 8-bit, $\text{maxCode} = 255$ and $C = 127$. For 2-bit, $\text{maxCode} = 3$ and $C = 1$. The center C is the code that represents the value zero, which is why the scheme is called *mid-rise symmetric*: codes below C are negative, codes above C are positive.

4.2 Block partitioning

The value stream is cut into consecutive blocks of B elements. Block j spans indices $[jB, \min((j+1)B, N))$; the final block may be short. The number of blocks is $\text{ceil}(N / B)$, and exactly one scale is stored per block. Block-wise (rather than per-tensor) scaling is the single most important quality decision in the format: it bounds the influence of any outlier weight to the 32 values that share its block.

4.3 Per-block scale

Within block j the encoder finds the maximum absolute value and divides it by the positive code budget to obtain the block's step size, or scale:

$$m_j = \max\{ |w_i| : i \text{ in block } j \}$$

$$s_j = m_j / C \text{ if } m_j > 0, \text{ else } s_j = 1$$

Dividing by C (not by maxCode) is what makes the representation symmetric: the most extreme weight in the block lands exactly C steps away from the center, i.e. at code $\text{maxCode}-1$ or at code 0 with no loss. An all-zero block is given the sentinel scale 1 to avoid a division by zero; its codes are all C and it dequantizes back to all zeros.

4.4 The code map

Each weight is divided by its block scale, rounded to the nearest integer, shifted by the center, and clamped into the representable range:

$$q_i = \text{clamp}(\text{round}(w_i / s_j) + C, 0, \text{maxCode})$$

The clamp guards the rare case where rounding pushes a value one step past the budget. Because s_j is defined from the block maximum, the pre-clamp code is already within range for the extreme values; the clamp matters only for intermediate values after rounding and is a no-op for the vast majority of weights.

4.5 The rounding rule

Rounding is **round-half-away-from-zero**, implemented as truncation after a signed half-step. This exact rule is part of the contract: a decoder that rounds differently would produce different codes and break bit-for-bit parity across implementations.

$$\text{round}(x) = \text{trunc}(x + 0.5) \text{ if } x \geq 0$$

$$\text{round}(x) = \text{trunc}(x - 0.5) \text{ if } x < 0$$

The intermediate is also saturated to the 32-bit signed range before the shift, so a non-finite or absurd input cannot wrap; in practice the encoder additionally rejects any non-finite weight outright, because a NaN in a weight tensor indicates a corrupt checkpoint.

4.6 Dequantization

Decoding is the exact algebraic inverse. For element i in block j :

$$\hat{w}_i = (q_i - C) \times s_j$$

There is no zero-point table and no per-element metadata: the only side information is one scale per block. This is why the decode path is trivially vectorizable and why it can be implemented identically in Go, in C++, and in a GPU shader.

4.7 Error analysis

The quantizer is a uniform mid-rise quantizer with step s_j . For any weight that is not clamped, the reconstruction error is bounded by half a step, and — modelling the rounding residual as uniform on the step interval — the mean squared error and signal-to-quantization-noise ratio follow the classical results:

$$|w_i - \hat{w}_i| \leq s_j / 2$$

$$\text{MSE} \approx s_j^2 / 12$$

$$\text{SQNR} \approx 6.02 \cdot b + 1.76 \text{ dB (full-scale signal)}$$

Two consequences drive the format's design. Each additional bit buys about 6 dB of fidelity, so the gap between 4-bit and 8-bit is roughly 24 dB — large enough that the most sensitive tensors are worth promoting (Section 7). And because s_j tracks the local block maximum, a tensor with a few large outliers and many small weights is quantized far more accurately block-by-block than it would be under a single global scale.

4.8 Worked example

Take a single 4-bit block ($b = 4$, so $\text{maxCode} = 15$, $C = 7$) of eight weights. The block maximum is 0.92, giving $s = 0.92 / 7 = 0.131$. The table shows each weight's code and its reconstruction.

Weight w	w / s	round	$+C \rightarrow \text{code } q$	clamp	decoded $(q-C) \cdot s$	error
0.92	7.02	7	14	14	0.918	0.002
-0.92	-7.02	-7	0	0	-0.918	0.002
0.31	2.37	2	9	9	0.262	0.048
0.05	0.38	0	7	7	0.000	0.050
-0.44	-3.36	-3	4	4	-0.393	0.047
0.68	5.19	5	12	12	0.655	0.025
-0.11	-0.84	-1	6	6	-0.131	0.021
0.00	0.00	0	7	7	0.000	0.000

Table 4.1 — A worked 4-bit block. Every error is at or below $s/2 = 0.065$, as the bound predicts; the extreme values ± 0.92 are recovered almost exactly.

5 · Bit-packing and memory layout

After every element has a code in $[0, \text{maxCode}]$, the codes are serialized into a dense bit stream. PAccel uses a **least-significant-bit-first** packing: code i occupies bits $[i \cdot b, (i+1) \cdot b)$, with the low bit of the code written to the low bit position, and codes are allowed to straddle byte boundaries.

5.1 General packing and the size formula

The general packer writes each of the b bits of each code in turn, advancing a global bit cursor. The packed payload size for the linear layout is therefore

$$\text{packedBytes} = \text{ceil}(N \times b / 8)$$

and the per-block scale table adds two bytes per block (Section 6), i.e. $2 \cdot \text{ceil}(N / B)$ bytes. For a 4-bit tensor with the default block size of 32, the scale overhead is 2 bytes per 32 weights — about 0.5 bits per weight — so the effective rate is close to 4.5 bits per weight, still nearly a 7× reduction from fp32.

5.2 Byte-aligned fast paths

The common widths are special-cased for speed, because they align to byte or nibble boundaries and avoid the per-bit loop: at 8 bits a code is one byte; at 4 bits two codes share a byte (low nibble first); at 2 bits four codes share a byte. These fast paths are pure performance — they produce exactly the same bytes the general packer would.

5.3 The v6 AVX-friendly tiled int4 layout

Package version 6 introduces an alternative packing for 4-bit, rank-2 weight matrices whose inner dimension is a multiple of the block size: the **output-tiled int4** layout. Instead of storing nibbles in plain row-major order, it groups the output columns into tiles of sixteen and interleaves the nibbles within a tile so that a single vector load pulls a contiguous tile of weights ready for a SIMD matrix multiply. The mapping from a logical element index to its byte and nibble is:

```
col = index / outputs output = index mod outputs
tile = output / 16 lane = output mod 16
byteIndex = (tile · cols + col) · 8 + lane / 2
nibble = low if lane even, else high
```

This reordering is a pure permutation of the same nibbles: every code appears exactly once, the packed size is unchanged at $N/2$ bytes, and decoding to logical order is the inverse permutation. The layout is opt-in per tensor and is selected either by an explicit request from the loader or by a name marker; tensors that are not eligible silently fall back to the linear layout. The format also records, in the record header, which layout was used so a decoder always knows how to read it.

6 · Scale storage and fp16 numerics

From package version 4 onward, block scales are stored as IEEE-754 **half precision** (binary16) rather than single precision. A scale is a single positive magnitude per 32 weights; half precision carries about three to four significant decimal digits, which is ample for a step size, and halving the scale table shrinks the dominant metadata cost of low-bit tensors. The reader chooses the scale width purely from the package version: two bytes for version ≥ 4 , four bytes for older packages.

6.1 The conversion algebra

The half-to-single expansion splits the 16-bit value into sign, exponent and mantissa and rebases the exponent from the binary16 bias of 15 to the binary32 bias of 127, handling the three special regimes explicitly:

Regime	Condition	Result
Zero	$\text{exp} = 0, \text{mantissa} = 0$	signed zero
Subnormal	$\text{exp} = 0, \text{mantissa} \neq 0$	normalize: shift mantissa left until the implied bit appears, adjust exponent

Regime	Condition	Result
Inf / NaN	$\text{exp} = 31$	$\text{sign} \cdot \infty$ if mantissa = 0, else NaN
Normal	$0 < \text{exp} < 31$	rebias exponent (+127–15), left-justify mantissa by 13 bits

Table 6.1 — Half-to-single decode regimes. The single-to-half encoder is the inverse, rounding the truncated mantissa to nearest and saturating overflow to infinity.

Because scales are strictly positive and bounded well within the half-precision normal range, the subnormal and infinity regimes essentially never occur for real models; they are handled only for completeness and robustness. The round-trip of any stored scale is exact to within the half-precision step, which the error budget of a low-bit quantizer absorbs without measurable quality change.

7 · Mixed-precision policy

Not all tensors are equally tolerant of aggressive quantization. A handful of them sit on the critical path of the model's output distribution, and quantizing them to 4 bits visibly degrades generation quality, while together they account for only a small share of the total bytes. PAccel therefore applies **smart mixed precision**: it raises these tensors to a floor of 8 bits while leaving the bulk transformer-block weights at the requested compact width.

7.1 Which tensors are protected

A tensor is treated as quality-sensitive if its (de-mangled, lower-cased) name contains any of a fixed set of markers, or if any of its dimensions exceeds 8192:

Category	Name markers	Why it is sensitive
Token embeddings	embed, wte, word_embeddings, tok_embeddings	Define the input/output token geometry; errors propagate to every layer
LM head	lm_head	Directly shapes the output logit distribution
Normalization	norm	Tiny tensors with high leverage on activations
Biases	bias	Small, additive, cheap to keep precise
Positional / rotary	rope, sin_cache, cos_cache, per_layer	Encode position; sensitive to step error
Very large dims	any dimension > 8192	Wide rows magnify per-element error in matmuls

Table 7.1 — The mixed-precision selection rule. Matched tensors are quantized at $\max(8, \text{requested})$ bits; all others use the requested width.

7.2 Name de-mangling

Because ONNX and OnnxStream mangle tensor names (encoding the dot in a path as `_2e` and an underscore as `_5f`), the matcher first reverses that mangling and collapses repeated underscores before testing the markers, so the policy fires identically whether weights arrive from safetensors, ONNX, or the OnnxStream manifest.

The selection is also used in the cost model that decides whether to compress a tensor at all: the estimated quantized size uses the *effective* bit width, so a sensitive tensor is weighed at its promoted width when

deciding compression versus raw storage (Section 10).

8 · Auxiliary codecs

8.1 Int8 activation quantization

For distributed inference, hidden-state activations are exchanged between shards in an **int8 wire format** that is distinct from the weight quantizer. It is per-tensor symmetric with a fixed zero point: a single fp32 scale equal to the tensor's maximum absolute value divided by 127 is written as a four-byte prefix, followed by one signed byte per element.

```
scale = max|v| / 127
q = clamp( round( v / scale ), -128, 127 )
v-hat = q × scale
```

This is deliberately the cheapest possible scheme — one multiply to decode — because it sits on the latency-critical path between shards. Empirically the mean error on LLM activations is around 0.4 percent, which the downstream layers tolerate. The same routine is provided for fp16-packed inputs, widening each half to single precision before quantizing.

8.2 Raw run-length encoding

Tensors that are not worth quantizing — integer index tables, boolean masks, tiny constants — are stored raw. When such a tensor is highly repetitive, an element-wise run-length codec is tried: the stream records the element width, the element count, a run count, and then (run-length, element) pairs. Encoding is abandoned if it fails to beat the raw size by a safety margin, so RLE never makes a tensor larger; the record header flags which of the two raw forms was used.

8.3 v7 transposed aliases for direct matmul

To let a runtime perform a weight-by-activation product without transposing weights at load time, the compressor can emit **pre-transposed aliases** of linear and embedding weights under reserved names. The alias is produced by transposing the source matrix and quantizing the transpose directly, so the runtime reads it in the orientation its kernel wants. Aliases are additive and opt-in; they trade a little package size for a faster, allocation-free matmul hot path, and a tied language-model head can be synthesized from the token embedding when the checkpoint omits it.

9 · The PACONNX1 container format

All multi-byte integers are little-endian. A package is a fixed header followed by one record per tensor. The reader accepts versions 5 and 6 fully and recognizes the version-4 scale width; it rejects anything newer than it understands.

9.1 Header

Field	Type	Meaning
magic	8 bytes	the ASCII signature PACONNX1
version	u32	package version (latest 6)

Field	Type	Meaning
bitWidth	u32	default quantization width for the package
blockSize	u32	default block size (typically 32)
modelSize	u64	size in bytes of the original source model
tensorCount	u32	number of records that follow

Table 9.1 — Package header (28 bytes after the signature).

9.2 Per-tensor record

Field	Type	Meaning
nameLen / name	u32 + bytes	UTF-8 tensor name
sourceDataType	u32	original element type (float32, float16, bf16, int64, ...)
encoding	u32	0 raw · 1 compressed · 2 raw+RLE · 3 compressed tiled-int4
rank / shape	u32 + rank·u64	tensor dimensions
sourcePayloadBytes	u64	original payload size (for ratio reporting)
– if compressed –		
tensorBitWidth	u32	effective width for this tensor (may exceed the package default)
tensorBlockSize	u32	block size for this tensor
layout / tileOutputs	u32 + u32	present only for encoding 3 (tiled int4)
scaleCount	u32	number of block scales
packedLen	u64	length of the packed code stream
scales	scaleCount · fp16	per-block scales (fp32 in packages < v4)
packed	packedLen bytes	the packed codes
– if raw / raw+RLE –		
rawLen / rawPayload	u64 + bytes	raw or RLE-encoded payload

Table 9.2 — Record layout. The compressed and raw branches are mutually exclusive, selected by the encoding field.

The format is self-contained and streamable: a reader walks records sequentially, and because each record declares its own lengths it can skip tensors it does not need. A cheap header-only read recovers the version, defaults, model size and tensor count without decoding any payload, which the runtime uses for capability gating and snapshot validation.

10 · The compression pipeline and algorithms

The compressor accepts three input shapes and normalizes all of them into the record model above.

Source	Detection	Parsing
ONNX model	protobuf stream	walk graph (field 7) → initializers (field 5); read dims, data type, raw or typed values
OnnxStream dir	a model.txt manifest	parse tensor specs and read each loose .bin weight file
safetensors	.safetensors files	read the JSON header, then each tensor's byte span by offset

Table 10.1 — The three ingest paths. A minimal protobuf and safetensors-header reader is built in; no protobuf library is linked.

For each tensor the pipeline is: decode the source bytes to float32 (widening fp16, bf16 or double as needed); choose the effective bit width from the mixed-precision policy; decide whether compression is worthwhile; then either block-quantize and pack, or store raw (with RLE if it helps). The compression decision is a simple inequality — compress only if the estimated quantized size plus a small per-record overhead is strictly smaller than the raw size:

```
quantizedBytes + overhead < rawBytes ⇒ compress
```

This keeps tiny constants raw while compressing every real weight matrix, including the large embedding and projection tensors. The work is parallelized on two levels — across tensors and, for very large tensors, across blocks — under a single CPU-worker budget so the two pools together stay near the machine's core count rather than oversubscribing. The quantizer is deterministic: the same input and parameters always produce byte-identical output, which is what makes cross-language parity testable.

11 · The bare-minimum port (the deliverable)

The engagement delivers a clean extraction of the format with the heavy dependencies removed. The guiding principle is that the format is pure arithmetic, so the port is small, auditable, and free of onnxruntime, protobuf, Eigen and abseil.

11.1 Why onnxruntime is unnecessary

In the original engine, onnxruntime was reached only through a thin dynamic-load bridge that supplied an optional accelerated matrix multiply. The format's encode and decode paths never call it. The port therefore drops the bridge entirely and ships a portable scalar matrix multiply in the extracted C++ core, so an integrator can dequantize and multiply with zero third-party dependencies, and can later swap in DirectML, ONNX Runtime, or a WebGPU kernel behind the same interface if they want hardware acceleration.

11.2 Components

Component	Language	Contents	Dependencies
libpaccel	Go	fp16, block quantize/dequantize, bit-pack incl. tiled int4, PACONNX1 reader/writer, safetensors ingest, int8 activations, RLE, mixed-precision policy	standard library only
accelerator	C++20	the same quantization algebra plus a portable scalar GEMM; bit-identical to the Go path	standard library only

Component	Language	Contents	Dependencies
paccelchat	Go	CLI frontend: pull from Hugging Face, compress to .paccel, inspect, and chat	libpaccel + stdlib

Table 11.1 — The three deliverable components. None links onnxruntime; the Go build is pure (no cgo).

11.3 The frontend

The frontend is a simple chatbot with Hugging Face download capabilities. It can pull a model from the Hub, compress it into a .paccel container with libpaccel, inspect a container, and run an interactive chat. The chat engine dequantizes the model's real token-embedding matrix through libpaccel and generates tokens by a nearest-neighbour walk in embedding space — a genuine, if minimal, use of the decoded weights that proves the codec round-trips real model tensors into usable floats. A full autoregressive transformer executor is intentionally out of scope for the bare-minimum port; the frontend exposes a single engine interface as the seam where such an executor — or the native C++ matmul path — plugs in, with libpaccel already supplying dequantized weights on demand.

12 · The V7 direct graph — the optimal inference route

Everything to this point describes how weights are stored. V7 is how they are best *run*. It is a packaging and execution mode that removes the last general-purpose layer between a quantized checkpoint and a forward pass: instead of exporting an ONNX graph and interpreting it, V7 stores the weights already in the orientation the math wants and pairs them with a tiny architecture recipe that a compact, hand-written decoder executes directly. For the transformer families it targets it is the shortest, lowest-overhead path to a token.

12.1 What V7 changes

Two things. **First**, every linear projection and the language-model head are stored **pre-transposed**. A framework weight is laid out as [outputs, inputs] and a matmul must implicitly transpose it; V7 stores it as [inputs, outputs] under a reserved name, so the runtime computes the product directly with no transpose and no scratch buffer. **Second**, the compute graph is not serialized at all — it is reconstructed from a small JSON recipe that records the architecture and its shapes. The package becomes a pair: a recipe and a direct-layout weight container.

Recipe field	Meaning
format / version / package_mode	power-paccel-direct · 7 · direct-only
graph_kind / model_type	transformer_decoder · qwen2 or qwen3 (Llama-family)
weight_layout	v7-transformer-direct-fused-int4 (fused q/k/v & gate/up, pre-transposed, tiled int4)
hidden_size, intermediate_size	model and MLP widths
num_hidden_layers	decoder block count
num_attention_heads / num_key_value_heads	grouped-query attention fan-out

Recipe field	Meaning
head_dim, vocab_size	per-head width and token count
rms_norm_eps, rope_theta, hidden_act	normalization, rotary base, activation (SiLU)
tie_word_embeddings	whether the head reuses the embedding

Table 12.1 — The model.v7.json recipe. These fields fully determine the decoder, so no graph needs to be shipped.

12.2 The graph it runs

The recipe drives a standard decoder-only transformer forward pass — the Qwen2/Qwen3, Llama-family architecture — expressed as a fixed sequence of primitive operations over the direct-layout weights:

```
h = Embed[token]
for each layer:
  a = RMSNorm(h) ; q,k,v = a·Wq, a·Wk, a·Wv (pre-transposed)
  apply RoPE(theta) to q,k ; grouped-query causal attention
  h = h + (attn)·Wo
  m = RMSNorm(h) ; h = h + ( SiLU(m·Wgate) * (m·Wup) )·Wdown
  h = RMSNorm(h) ; logits = h·Wlm_head
```

Each weight named with a dot product above is read in V7 orientation, so every $\cdot$ is a plain row-major multiply against an int4 (or mixed 8-bit) block-quantized matrix that libpaccel dequantizes on demand. RMSNorm, rotary position embedding, grouped-query attention and the SwiGLU feed-forward are the only operations; there is no operator zoo to interpret. Optional per-head query/key normalization (Qwen3) is applied when present.

12.3 Fused linear projections

The current V7 layout, v7-transformer-direct-fused-int4, goes one step further than pre-transposition. Within a block the three attention projections all read the same input — the normalized hidden state — and so do the two gate-and-up projections. Three (and two) separate matmuls against the same input are wasteful; V7 instead **fuses** them into a single matrix by concatenating their pre-transposed forms along the output axis. One product then computes all three projections at once, and the decoder slices the output columns back apart:

```
W_qkv = [ Wq^T | Wk^T | Wv^T ] shape [ hidden, out_q + 2·kv ]
x·W_qkv = [ x·Wq^T | x·Wk^T | x·Wv^T ] -> slice columns into q, k, v
W_gate_up = [ Wg^T | Wu^T ] shape [ hidden, 2·intermediate ]
```

The fusion is exact: because each weight is transposed and the parts are laid side by side, the single fused product is bit-identical to the three (or two) separate products, and a column slice recovers each one. Attention projection biases, where a model has them, are added to the slices. The payoff is one matmul where there were three, and one where there were two — higher arithmetic intensity, a single contiguous tiled-int4 weight to stream per pair, and fewer kernel launches — which is exactly the shape of work a wide SIMD or tensor-core unit rewards. Decoders that prefer the split tensors can still request them; the loader reconstructs a fused matrix from split parts, or splits a fused one, as needed.

12.4 Why it is optimal

V7 wins on every axis that matters for local inference. It removes the ONNX export and the interpreter, so there is no graph to parse and no general-purpose runtime to load. It removes the runtime transpose and the buffers that go with it, so the matmul reads weights straight from the mapped package. It keeps the v6 tiled int4 layout, so each matmul streams vector-friendly tiles. And it preserves the mixed-precision policy, so the embedding, head and norms stay at eight bits while the transformer blocks ride at four. The result is the smallest resident footprint and the fewest instructions per token of any route the format supports — which is exactly what a memory-bound, on-device accelerator rewards, as the next section shows.

The deliverable includes V7 in full: the alias builder that produces the pre-transposed, tiled tensors; the recipe writer; a loader that binds a recipe and package into a runnable model; and a dependency-free Go executor of the graph above, verified end-to-end on a synthetic model.

13 · NVIDIA DGX Spark — mixed CPU/GPU advantages

The V7 route and NVIDIA's GB10-class hardware are unusually well matched. NVIDIA's DGX Spark is a desk-side personal AI computer built on the GB10 Grace Blackwell Superchip: a 20-core Arm CPU (ten Cortex-X925 performance cores plus ten Cortex-A725 efficiency cores) and a Blackwell GPU on one package, joined by NVLink-C2C and sharing 128 GB of LPDDR5X as a single **coherent** memory pool. That coherent, unified memory — not a discrete VRAM island behind PCIe — is the architectural fact that V7 is built to exploit.

DGX Spark (GB10) attribute	Figure	Why it matters for V7
Unified memory	128 GB LPDDR5X, CPU+GPU coherent	a single weight image, read by either processor with no copy
Memory bandwidth	≈ 273 GB/s shared	decode is bandwidth-bound: 4-bit weights ≈ 4× the tokens/s of fp16
Interconnect	NVLink-C2C (not PCIe)	≈ 5× PCIe gen5; no host-to-VRAM staging
GPU math	Blackwell, 5th-gen Tensor Cores, NVFP4	native 4-bit GEMM aligns with PAccel int4 blocks
Compute	up to 1 PFLOP FP4 (sparse) / 1000 TOPS	ample for batch-1 decode once memory is the limit
CPU	20 Arm cores up to 4 GHz	runs the 8-bit sensitive tensors while the GPU runs the 4-bit blocks

Table 13.1 — GB10 / DGX Spark attributes and their V7 leverage. Figures are NVIDIA's published GB10 specifications.

13.1 Coherent memory makes the mixed CPU/GPU split free

On a discrete GPU, splitting a model across CPU and GPU means copying tensors across PCIe every step — usually a loss. On GB10 the CPU and GPU address the **same** physical 128 GB through NVLink-C2C with cache coherence, so a tensor written by one is visible to the other with no transfer. V7 turns this into a clean division of labour: the quality-sensitive eight-bit tensors the format already singles out — token embedding, LM head, normalization — are exactly the latency-light, gather-heavy operations that suit the 20 Arm cores, while the four-bit transformer blocks are the dense GEMMs that belong on the Blackwell tensor cores. Because V7 stores one pre-transposed weight image and both processors read it in place, the split costs nothing in data movement.

13.2 Memory footprint is the lever, and V7 pulls it hardest

A 273 GB/s pool is generous in capacity but modest in bandwidth next to a discrete GPU's terabytes per second, so single-stream decoding is bound by how many weight bytes must be read per token. Cutting weights from sixteen to four bits cuts that traffic roughly fourfold and lifts token throughput in proportion; keeping the whole model resident in 128 GB avoids the far larger penalty of spilling. V7 is the configuration that minimizes both resident bytes and per-token bytes at once, which is why it is the right default on this class of machine.

13.3 PAccel int4 and NVFP4 are the same idea

Blackwell's native four-bit format, NVFP4, groups values into **blocks of sixteen**, each with its own higher-precision (FP8) block scale plus a per-tensor scale, averaging about 4.5 bits per value. PAccel's block-wise int4 with a per-block fp16 scale is the same construction, and its v6 output tile is sixteen lanes wide — so a V7 tile maps almost one-to-one onto an NVFP4 micro-block. In practice the engine can hand V7 tiles to the Blackwell tensor cores with only a scale re-encoding, letting hardware apply the block scale, accumulate the dot products in high precision, and return FP32 results. The format was designed around block-scaled low precision; the hardware now implements it natively, and V7 is the path that lines them up.

14 · Reference architecture and Microsoft integration

PAccel is designed to drop into the Windows on-device inference stack as the weight layer beneath whatever execution provider a team prefers. The container is the portable artifact; the dequantized tensors it yields can be consumed by a CPU kernel, by DirectML, by ONNX Runtime, or by a WebGPU path, all behind one interface.

Layer	Role	Microsoft mapping
Acquisition	fetch weights	Hugging Face Hub pull; enterprise model registry
Container	store quantized weights	the .paccel file; libpaccel reader/writer
Decode	codes → float tensors	libpaccel (Go) or the C++ accelerator
Execute	matmul / attention	portable kernel now; DirectML / ONNX Runtime / WebGPU later
Frontend	chat UX	WinUI / .NET app over the engine interface; MSIX packaging

Table 14.1 — Layered architecture. PAccel owns the container and decode layers and stays neutral about the execution provider.

Because decode is dependency-free and deterministic, the same container produces identical weights on x64 and ARM64, on Windows, Linux and macOS, which simplifies certification and reproducible builds. The C++ core compiles with any C++20 toolchain, including MSVC, and its CMake target is a drop-in for the existing compressor so an integrator can adopt it incrementally.

15 · Verification and quality

The format's value depends on numeric correctness, so the deliverable is gated by an executable test suite that proves the algebra rather than asserting it. Four invariants are checked in every supported language.

Invariant	Statement	How it is checked
Round-trip bound	decode error $\leq s/2$ per element	quantize/dequantize synthetic and real tensors at 2/4/8 bits
Packing bijection	<code>unpack(pack(codes)) = codes</code>	exhaustive over widths 2–8
fp16 round-trip	stored scales survive within fp16 step	representative and random values
Tile-map bijection	tiled int4 is a permutation of nibbles	every nibble covered exactly once

Table 15.1 — Proven invariants. The same checks run in Go and in C++, plus an independent Python reference that re-derives the algebra from this specification.

The C++ accelerator and the independent Python reference have both been compiled and run during the preparation of this proposal and pass every check; the Go library mirrors the same algebra line-for-line. Beyond these invariants, the plan adds property-based fuzzing of the container reader against malformed inputs, a determinism check (identical bytes across repeated runs and across platforms), and a cross-language parity gate that compresses a tensor in Go and decompresses it in C++ and vice versa. A small set of real Hugging Face checkpoints serves as end-to-end regression fixtures.

16 · Delivery plan and timeline

The work is organized as eight weekly milestones, each with a concrete exit criterion. The specification and parity oracle come first so that every later component is validated against a fixed contract.

Wk	Milestone	Exit criterion
1	Discovery & spec freeze	byte layout and algebra locked; parity oracle defined
2	libpaccel core	fp16 + quantize/dequantize + packing pass unit tests
3	Container & ingest	PACONNX1 v4–6 read/write; safetensors, RLE, int8 codecs
4	C++ accelerator + parity	C++ core compiles on MSVC/Clang/GCC; cross-language parity green
5	Frontend + HF download	pull / compress / info commands working on real repos
6	Chat engine	embedding-grounded chat over a downloaded model
7	Verification & hardening	fuzzing, determinism, x64/ARM64 + Win/Linux/macOS, perf pass
8	Packaging & handoff	MSIX demo, documentation, Microsoft walkthrough

Table 16.1 — Eight-week milestone plan.

Risk is front-loaded: the only genuinely novel work — re-deriving the exact byte layout and the tiled-int4 permutation and proving parity — happens in weeks 1 and 4, leaving the back half for hardening and packaging. The plan assumes a single AI-augmented senior engineer with part-time QA, which the budget reflects.

17 · Budget

The engagement is a fixed fee of **USD \$10,000**. The structure assumes a heavily AI-augmented solo engineer, which is why a complete, tested port and specification is achievable at this figure: the model carries much of the boilerplate, test synthesis and documentation drafting, while the engineer owns architecture, the exact-parity work, and review. The budget splits into human labour and AI/infrastructure.

Human power

Workstream	Hours	Rate	Cost
Format port + libpaccel (Go): algebra, container, ingest	55	\$65	\$3,575
C++ accelerator extraction + cross-language parity	20	\$65	\$1,300
Frontend chatbot + Hugging Face download	30	\$60	\$1,800
Verification, fuzzing, cross-platform, documentation	20	\$55	\$1,100
Human subtotal	125		\$7,775

AI tooling and infrastructure

Item	Basis	Cost
LLM coding + eval / test synthesis (Claude Code & API)	2 months of assisted development	\$1,025
CI compute + Windows x64/ARM64 cross-build runners	GitHub Actions / cloud runners	\$600
Hugging Face bandwidth + test-model storage	fixture checkpoints	\$250
Copilot / editor assistant seats	2 months	\$50
Contingency	≈ 3%	\$300
AI & infrastructure subtotal		\$2,225
Total fixed fee		\$10,000

Table 17.1 — Line-item budget. Human labour is 78% of the fee; AI tooling and cross-platform CI make up the remainder.

Two assumptions are worth stating. The rates are blended contractor rates for AI-augmented delivery, not full-time-equivalent salary loaded cost; and the scope is the bare-minimum port and specification exactly as described, with a full hardware-accelerated executor and a production UI treated as separate, follow-on engagements.

18 · Risks and mitigations

Risk	Likelihood	Mitigation
Byte-layout drift from the source engine	Low	Lock the layout in week 1 against a golden package; gate every build on header parity
Rounding/parity mismatch across languages	Medium	Single specified rounding rule; cross-language parity test in CI

Risk	Likelihood	Mitigation
Sensitive-tensor list incomplete for a new model family	Medium	Policy is data-driven and extensible; add markers without format change
Hugging Face API or auth changes	Low	Isolate Hub access behind one module; support tokens and mirror endpoints
Scope creep toward a full executor	Medium	Engine interface fixes the boundary; executor is a separate engagement

Table 18.1 — Principal risks. The dominant technical risk — numeric parity — is directly addressed by the test architecture in Section 15.

19 · IP, licensing and security

The port is a clean extraction of the quantization algebra and container, written against the standard library in Go and C++ with no third-party runtime, which keeps the dependency surface — and therefore the audit and certification burden — minimal. There is no GPL or copyleft dependency in the deliverable. Model downloads honor Hugging Face authentication and can be pointed at an enterprise mirror, and the container reader is hardened against malformed inputs through bounds checks and property-based fuzzing so that loading an untrusted package cannot corrupt the host process.

Deliverable artifacts — source, tests, the specification, and a packaged demo — are handed over with documentation sufficient for a Microsoft platform team to own the code, including the exact algebra in this document so the numeric contract is reproducible from first principles.

20 · The compute engine and native multimodal engines

The same dependency-free philosophy extends past text. The native audio, image, and video engines all run on one small **compute engine** and one **diffusion core**, both of which the deliverable ports as standalone, cgo-free Go that links neither onnxruntime nor any BLAS.

20.1 The compute engine

The compute engine is a cooperative executor: it exposes the handful of float32 primitives every diffusion or transformer backbone needs — a biased matrix multiply, fused scaled-dot-product attention, and the elementwise SAXPY/add/scale — and splits each operation into device segments that run concurrently. Upstream those segments fan out across the CPU and every Metal GPU; this standalone build keeps the identical API and the cooperative plan machinery and ships a CPU-only backend, so it builds anywhere with no cgo and a GPU backend can be reintroduced without changing a single caller. The fused attention kernel is the shared hot path of every native diffusion transformer:

```
MatMulBias: out[r,c] = lhs[r,:] . rhs[c,:] + bias[c] (rhs is the [out,in] weight)
Attention: out[i,:] = softmax_j( scale * q[i,:] . k[j,:] ) . v[j,:]
```

20.2 The diffusion core

Above the compute engine sits the sampler algebra. Weights are quantized with the same PAccel format; only the schedule and the update rule differ by modality. Two samplers cover all three engines. The

Euler/Karras sampler with v-prediction (audio and video) preconditions the latent and takes an Euler step along the Karras sigma schedule; the few-step LCM sampler with epsilon-prediction (image) jumps between sub-sampled timesteps. Both reduce to the predicted clean latent on the final step:

```
Euler (v-pred): x0 = v*(-s/sqrt(s^2+1)) + x/(s^2+1)
x' = x + ((x - x0)/s) * (s_next - s)
LCM (epsilon): x0 = ( x_t - sqrt(1-a_t)*eps ) / sqrt(a_t)
x_prev = sqrt(a_prev) * x0
Guidance: out = uncond + scale*(cond - uncond)
```

A single generic loop drives both: scale the initial noise by the schedule's start sigma, then per step precondition the latent, run the backbone (twice when classifier-free guidance is on), apply the sampler update, and finally decode. The backbone and the decoder are interfaces, so the heavy production modules — the Stable Audio 3 diffusion transformer, the image UNet, the CogVideoX spatio-temporal transformer, and their VAE decoders — load from a .paccel through libpaccel and drop into the same seams a compact reference backbone occupies today.

Engine	Pipeline	Sampler	Latent
Audio	T5Gemma → DiT → VAE (Stable Audio 3)	Euler v-prediction	frames × channels
Image	CLIP/T5 → UNet → VAE	LCM epsilon (1–8 steps)	H/8 × W/8 grid
Video	T5 → 3D DiT → 3D VAE (CogVideoX)	Euler v-prediction	frames/4 × H/8 × W/8

Table 20.1 — The three native multimodal engines share one compute engine and one diffusion core; only the pipeline, sampler, and latent geometry differ.

Every layer here is the same trade the format makes for text: a tiny, auditable, dependency-free core with an exactly specified numeric contract, sized to run on the coherent unified memory of a Copilot+ or DGX Spark-class device.

Appendix A · Symbols, worked data, and manifest

A.1 Symbols

Symbol	Meaning
b	bit width, 2–8
B	block size (default 32)
N	number of weights in a tensor
maxCode	$2^b - 1$, the largest code
C	$\text{floor}(\text{maxCode} / 2)$, the center / implicit zero point
m _j	max absolute weight in block j
s _j	block scale, m _j / C
q _i	the stored code for weight i

A.2 Reductions achieved (illustrative, per weight)

Profile	Bits/weight (codes)	Scale overhead	Effective rate	vs fp32	vs fp16
8-bit, B=32	8.0	0.5	8.5	3.8×	1.9×
4-bit, B=32	4.0	0.5	4.5	7.1×	3.6×
2-bit, B=32	2.0	0.5	2.5	12.8×	6.4×

Table A.1 — Effective bit rate including the fp16 scale table at the default block size. Mixed precision raises sensitive tensors to 8-bit, so a real model lands between its nominal profile and 8-bit.

A.3 Deliverable file manifest

Path	Purpose
Caveats/paccel-format.pdf	this specification and proposal
Caveats/libpaccel/	the Go format library (no onnxruntime)
Caveats/v7graph/	the V7 direct graph – the optimal inference route
Caveats/paccelchat/	the chatbot frontend with Hugging Face download
Caveats/compute/	the cooperative compute engine (CPU, no cgo)
Caveats/diffusion/	samplers (Euler/LCM/CFG) + latent-diffusion loop
Caveats/{audio,image,video}/	standalone multimodal engines + mmengine backbone
Caveats/cpp/	the extracted minimal C++ accelerator
Caveats/README.md, Makefile	build and demo instructions

Prepared for Microsoft. The algebra in Sections 4–9 fully determines the format; an implementation that reproduces it is bit-compatible with PAccel packages.